

1 Описание предмета тестирования

Сайт: Okko (<https://okko.tv/>) – крупнейший российский онлайн-кинотеатр, предоставляющий доступ к тысячам фильмов, сериалов, мультфильмов, телеканалов и спортивных трансляций.

Основные функции пользователя:

- поиск контента по ключевым словам, жанрам, актёрам;
- просмотр карточки фильма/сериала (описание, трейлер, рейтинг, отзывы);
- просмотр трейлеров и бесплатного контента;
- добавление фильмов в список «Запомнить»;
- покупка или аренда контента, оформление подписки;
- просмотр контента в плеере (в том числе с функциями паузы, перемотки);
- управление профилем и настройками аккаунта.

2 Описание окружения тестирования

- Тип устройства: ноутбук
- Процессор: Intel Core i5
- ОЗУ: 8 Гб
- ОС: Windows 11
- Разрешение: 1920x1080
- Браузер: Google Chrome
- Интернет: Wi-Fi

3 Сценарии использования

Сценарий 1 (позитивный): поиск фильма и добавление в список «Запомнить»

Цель: найти конкретный фильм и сохранить его для последующего просмотра.

Предусловие: пользователь авторизован на сайте.

Таблица 1 – Этапы первого сценария

№	Действие	Ожидаемый результат
1	Открыть главную страницу сайта okko.tv	Загружается главная страница с поисковой строкой, баннерами и рекомендациями
2	Ввести в поисковую строку запрос «Бойцовский клуб» и нажать Enter	Открывается страница результатов поиска, отображаются фильмы и сериалы, соответствующие запросу (в т.ч. «Бойцовский клуб»)
3	Выбрать фильм «Бойцовский клуб» из списка, кликнуть по нему	Открывается карточка фильма с подробным описанием, трейлером, кнопками «Смотреть», «Запомнить», «Просмотрено», «Не интересно», «Поделиться»
4	Нажать кнопку «Запомнить» (иконка «закладка»)	Кнопка меняет состояние на «В списке», появляется всплывающее уведомление о добавлении фильма в список «Запомнить»
5	Перейти в раздел «Моё» → «Запомненные»	В списке отображается добавленный фильм «Бойцовский клуб»

Листинг кода(из Selenium IDE)

```
import pytest
import time
import json
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.common.action_chains import ActionChains
from selenium.webdriver.support import expected_conditions
from selenium.webdriver.support.wait import WebDriverWait
from selenium.webdriver.common.keys import Keys
from selenium.webdriver.common.desired_capabilities import DesiredCapabilities

class TestNewTest():

def setup_method(self, method):
    self.driver = webdriver.Chrome()
    self.vars = {}

def teardown_method(self, method):
    self.driver.quit()

def test_newTest(self):
    self.driver.get("https://okko.tv/")
    self.driver.find_element(By.CSS_SELECTOR, ".OSSozSZ_ .JIJy8E_D").click()
    self.driver.find_element(By.CSS_SELECTOR, ".OSSozSZ_ .JIJy8E_D").send_keys("бойцовский клуб")
    self.driver.find_element(By.CSS_SELECTOR, ".OSSozSZ_:nth-child(2) .NlqD4Fua:nth-child(1) .Oce5ukhe:nth-child(2) .iSn2BQUn:nth-child(1)").click()
    self.driver.find_element(By.CSS_SELECTOR, ".MbmjcbEA:nth-child(2) .\\_9vY2xAN").click()
    element = self.driver.find_element(By.CSS_SELECTOR, ".MbmjcbEA:nth-child(2) > .HtTEYg53")
    actions = ActionChains(self.driver)
    actions.move_to_element(element).perform()
    self.driver.find_element(By.CSS_SELECTOR, ".MbmjcbEA:nth-child(2) .\\_9vY2xAN").click()
    element = self.driver.find_element(By.CSS_SELECTOR, ".MbmjcbEA:nth-child(3) > .HtTEYg53")
    actions = ActionChains(self.driver)
    actions.move_to_element(element).perform()
    self.driver.find_element(By.LINK_TEXT, "Moë").click()
    element = self.driver.find_element(By.CSS_SELECTOR, ".iSn2BQUn")
    actions = ActionChains(self.driver)
    actions.move_to_element(element).perform()
```

Сценарий 2 (позитивный): просмотр трейлера фильма без регистрации

Цель: проверить возможность просмотра трейлера без необходимости входа в аккаунт.

Таблица 2 – Этапы второго сценария

№	Действие	Ожидаемый результат
1	Открыть главную страницу сайта	Загружается главная страница

№	Действие	Ожидаемый результат
2	Ввести в поисковую строку запрос «Такси» и нажать Enter	Открываются результаты поиска
3	Выбрать фильм «Такси»	Открывается карточка фильма
4	Нажать кнопку «Трейлер» (или иконку воспроизведения трейлера)	Начинается воспроизведение трейлера в плеере на той же странице; видео и звук работают корректно, доступны элементы управления плеером
5	Остановить воспроизведение кнопкой «Пауза»	Воспроизведение приостанавливается, плеер остаётся на экране

Листинг кода(из Selenium IDE)

```
import pytest
import time
import json
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.common.action_chains import ActionChains
from selenium.webdriver.support import expected_conditions
from selenium.webdriver.support.wait import WebDriverWait
from selenium.webdriver.common.keys import Keys
from selenium.webdriver.common.desired_capabilities import DesiredCapabilities
```

```
class TestTest2():
    def setup_method(self, method):
        self.driver = webdriver.Firefox()
        self.vars = {}

    def teardown_method(self, method):
        self.driver.quit()

    def test_test2(self):
        self.driver.get("https://okko.tv/movie/taxi-975070")
        self.driver.set_window_size(962, 693)
        self.driver.find_element(By.CSS_SELECTOR, ".FtMhtc4s .\\_9vY2xAN").click()
        self.driver.execute_script("window.scrollTo(0,0)")
        element = self.driver.find_element(By.CSS_SELECTOR, ".zUEZkIiY")
        actions = ActionChains(self.driver)
        actions.move_to_element(element).perform()
        element = self.driver.find_element(By.CSS_SELECTOR, "body")
        actions = ActionChains(self.driver)
        actions.move_to_element(element, 0, 0).perform()
        self.driver.find_element(By.CSS_SELECTOR, ".zUEZkIiY svg").click()
        self.driver.find_element(By.CSS_SELECTOR, ".zUEZkIiY svg").click()
        self.driver.find_element(By.CSS_SELECTOR, ".h2hfNlw1 > .b_00enV_").click()
```

```

self.driver.find_element(By.CSS_SELECTOR, ".h2hfN1w1 > .b_00enV").click()
element = self.driver.find_element(By.CSS_SELECTOR, ".h2hfN1w1 > .b_00enV")
actions = ActionChains(self.driver)
actions.move_to_element(element).perform()
element = self.driver.find_element(By.CSS_SELECTOR, "body")
actions = ActionChains(self.driver)
actions.move_to_element(element, 0, 0).perform()
self.driver.find_element(By.CSS_SELECTOR, ".\\_8cp28QVr svg").click()
self.driver.execute_script("window.scrollTo(0,422.3999938964844)")

```

Сценарий 3 (негативный): попытка просмотра платного контента без активной подписки

Цель: проверить реакцию системы при попытке доступа к контенту, требующему подписки или аренды, у пользователя без подписки.

Таблица 3 – Этапы третьего сценария

№	Действие	Ожидаемый результат
1	Открыть главную страницу сайта	Загружается главная страница
2	Найти фильм, доступный только по подписке или за плату (например, «Чернобыль», если он помечен как требующий подписки)	Открывается карточка фильма, на которой присутствует кнопка «Оформить подписку»
3	Нажать кнопку «Оформить подписку» или «Улучшить подписку»	Появляется модальное окно с предложением оформить подписку или войти в аккаунт (если пользователь не авторизован); если пользователь авторизован, но подписка не активна – предлагается выбрать тариф. В любом случае доступ к просмотру фильма не предоставляется
4	Закрыть модальное окно (клик на крестик или вне окна)	Окно закрывается, пользователь возвращается к карточке фильма, просмотр не начинается

Листинг кода(из Selenium IDE)

```

import pytest
import time
import json
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.common.action_chains import ActionChains
from selenium.webdriver.support import expected_conditions

```

```

from selenium.webdriver.support.wait import WebDriverWait
from selenium.webdriver.common.keys import Keys
from selenium.webdriver.common.desired_capabilities import DesiredCapabilities

class TestTest3():
    def setup_method(self, method):
        self.driver = webdriver.Firefox()
        self.vars = {}

    def teardown_method(self, method):
        self.driver.quit()

    def test_test3(self):
        self.driver.get("https://okko.tv/serial/chernobyl")
        self.driver.set_window_size(969, 824)
        self.driver.find_element(By.CSS_SELECTOR, ".pLbFsg0W").click()
        element = self.driver.find_element(By.CSS_SELECTOR, ".gmlAgFvx .\\_9vY2xAN")
        actions = ActionChains(self.driver)
        actions.move_to_element(element).perform()
        self.driver.find_element(By.CSS_SELECTOR, ".gmlAgFvx .\\_9vY2xAN").click()
        self.driver.execute_script("window.scrollTo(0,0)")
        self.driver.find_element(By.CSS_SELECTOR, ".css-tpp7ox > span").click()
        element = self.driver.find_element(By.CSS_SELECTOR, ".css-tpp7ox > span")
        actions = ActionChains(self.driver)
        actions.move_to_element(element).perform()
        element = self.driver.find_element(By.CSS_SELECTOR, "body")
        actions = ActionChains(self.driver)
        actions.move_to_element(element, 0, 0).perform()

```

4 Результат ручного тестирования

Сценарий 1

Главная страница – скриншот с поисковой строкой. Результат: страница загружена, поиск доступен.

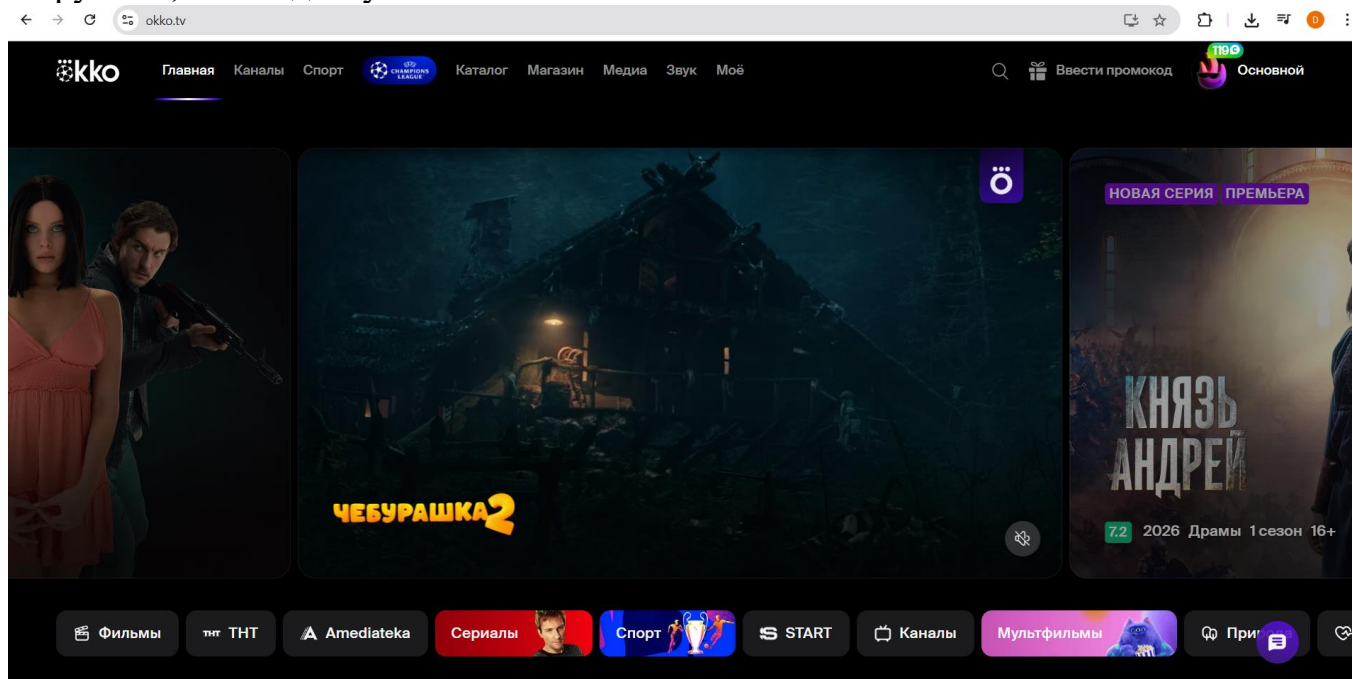


Рисунок 1 – Скриншот сайта с поисковой строкой.

Результаты поиска – скриншот списка найденных фильмов по запросу «Бойцовский клуб». Результат: поиск выполнен, фильмы отображаются.

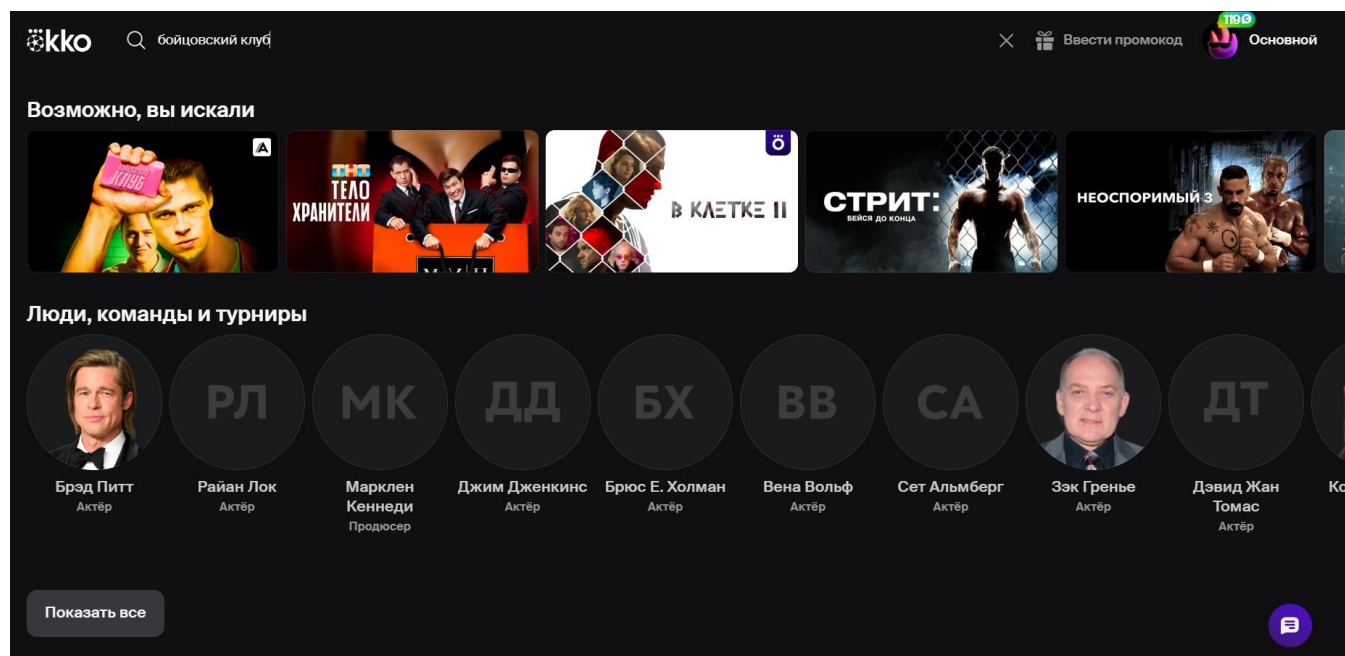


Рисунок 2 – Скриншот страницы результатов поиска

Карточка фильма – скриншот с описанием, «Смотреть», «Запомнить», «Просмотрено», «Не интересно», «Поделиться». Результат: карточка открыта, информация корректна.

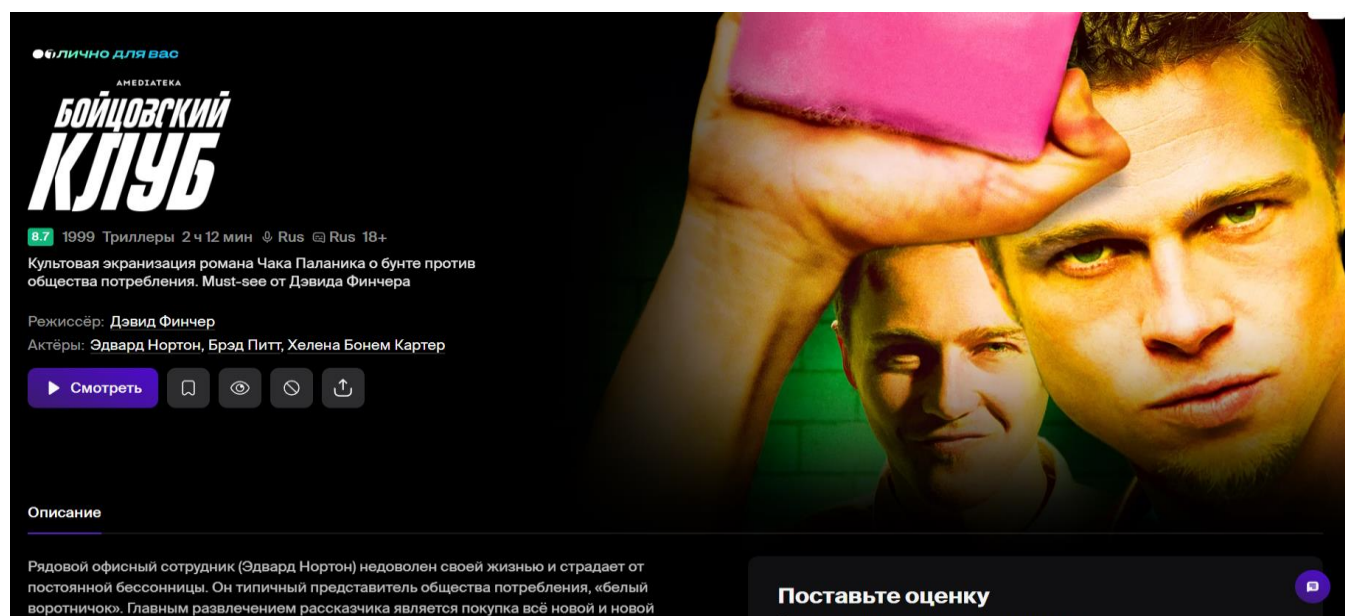


Рисунок 3 – Скриншот карточки фильма

Подтверждение добавления в список – скриншот с изменённой кнопкой («Запомнить») и всплывающим уведомлением.

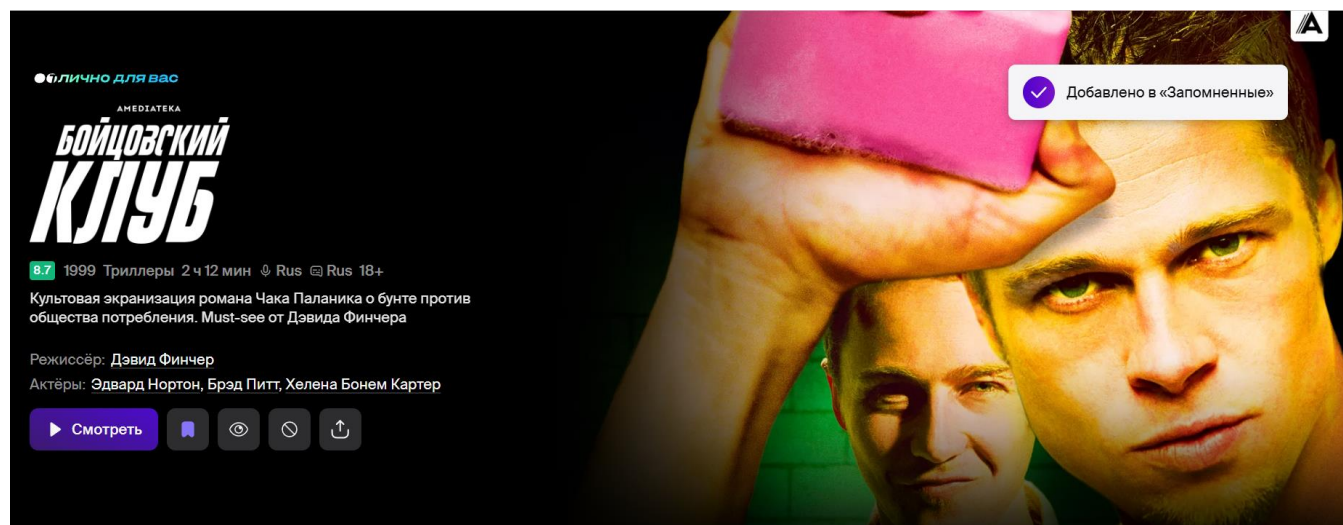


Рисунок 4 – Скриншот карточки после добавления.

Раздел «Посмотреть позже» – скриншот содержимого списка. Результат: добавленный фильм присутствует.

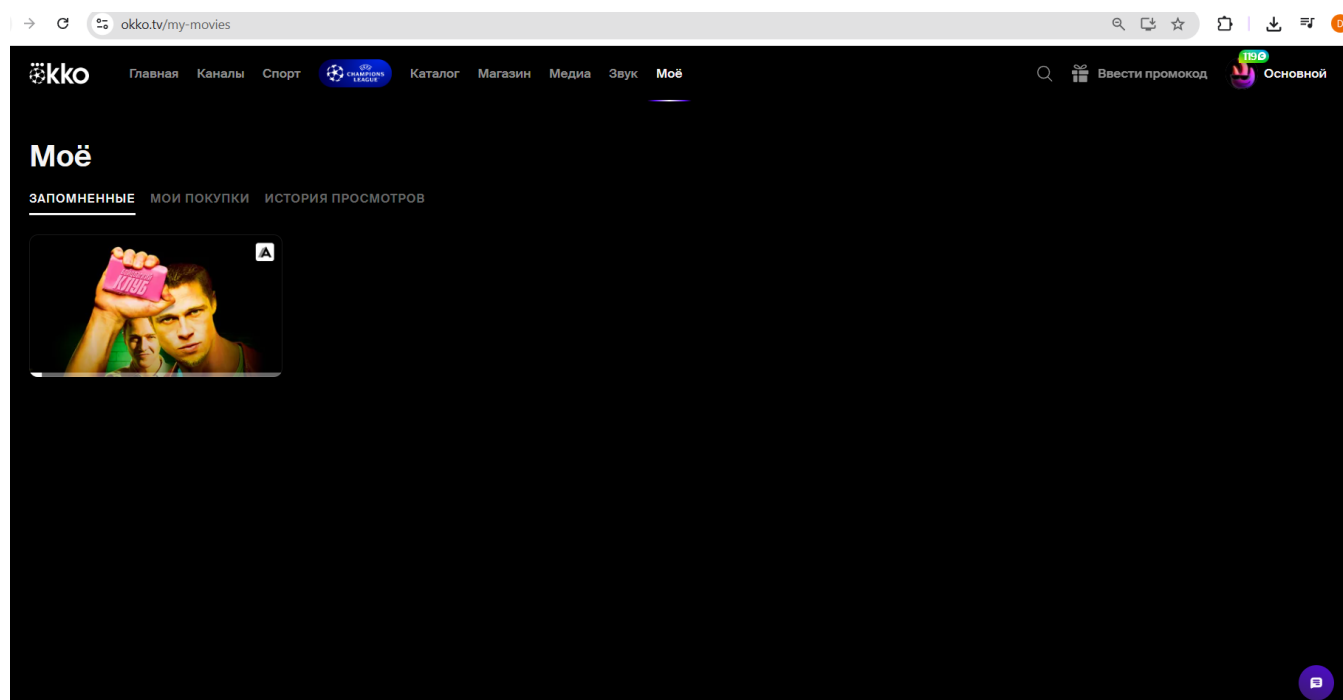


Рисунок 5 – Скриншот раздела «Запомненные».

Running 'test1'

1. open on /movie/fight-club OK
2. setWindowSize on 683x824 OK
3. click on css=.MbmjcbEA:nth-child(2) ._9vY2xAN OK
4. click on css=.MbmjcbEA:nth-child(2) ._9vY2xAN OK
5. click on css=.__9lww5cH_ .Vply81jm OK
6. mouseOver on css=.__9lww5cH_ .Vply81jm OK
7. mouseOut on css=.__9lww5cH_ .Vply81jm OK
8. click on css=.eM4Ndc70:nth-child(9) > .cO_8eiNN > span OK
9. click on css=.fLAz9yTN:nth-child(1) span OK

'test1' completed successfully

Рисунок 6 – Выполнение первого сценария.

Сценарий 2

Карточка фильма с выделенной кнопкой трейлера – скриншот момента перед нажатием. (При этом не выполнена авторизация) Результат: кнопка активна.

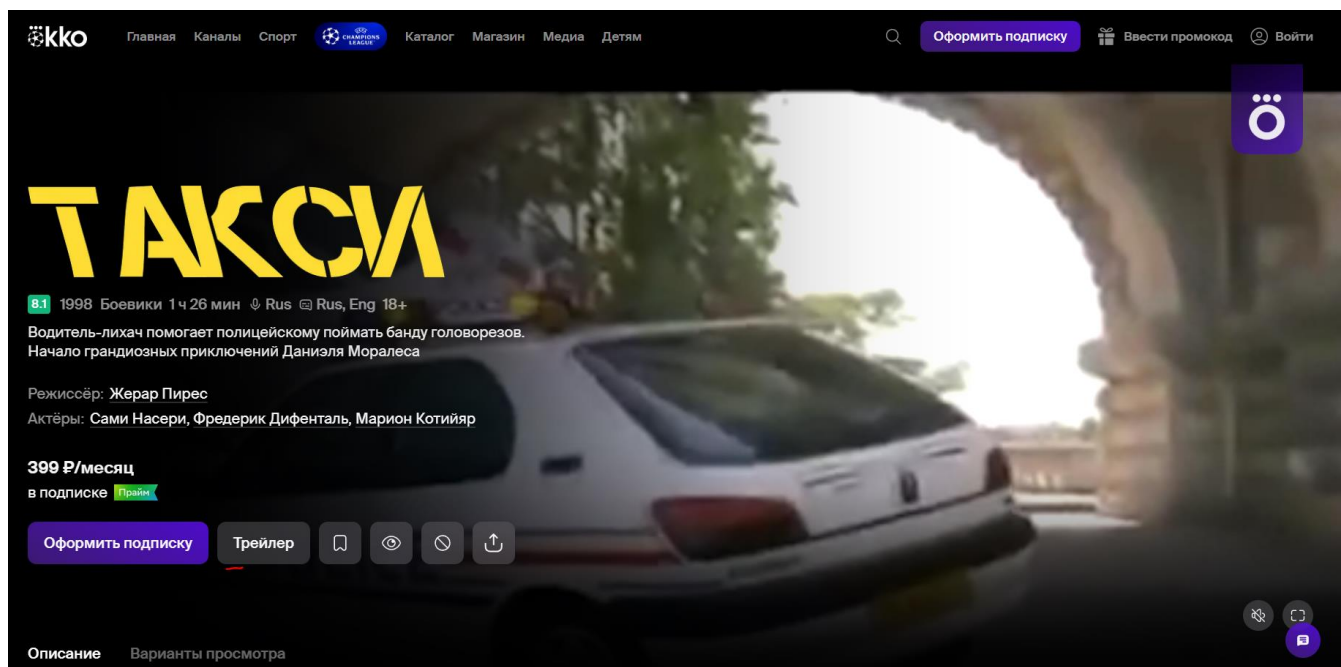


Рисунок 7 – Скриншот карточки с кнопкой «Трейлер».

Плеер с воспроизведением трейлера – скриншот экрана во время просмотра. Результат: видео запущено, элементы управления видны.



Рисунок 8 – Скриншот плеера с трейлером

Остановка воспроизведения – скриншот плеера на паузе. Результат: видео приостановлено.



Рисунок 9 – Скриншот оформления заказа (без оплаты).

```
Running 'test2'  
1. open on /movie/taxi-975070 OK  
2. setWindowSize on 962x693 OK  
3. click on css=.F1Mhtc4s\_9vY2xAN OK  
4. runScript on window.scrollTo(0,0) OK  
5. Trying to find css=zUEZkiiY... OK  
6. mouseOut on css=zUEZkiiY OK  
7. click on css=zUEZkiiY svg OK  
8. click on css=zUEZkiiY svg OK  
9. click on css=.h2hN1w1 > .b_O0enV_ OK  
10. click on css=.h2hN1w1 > .b_O0enV_ OK  
11. mouseOver on css=.h2hN1w1 > .b_O0enV_ OK  
12. mouseOut on css=.h2hN1w1 > .b_O0enV_ OK  
13. click on css=\\_8cp28QVr svg OK  
14. runScript on window.scrollTo(0,422.3999938964844) OK  
'test2' completed successfully
```

Рисунок 10 – Выполнение первого сценария.

Сценарий 3

Карточка фильма, требующего подписки – скриншот с кнопкой «Улучшить подписку». Результат: кнопка отображается.

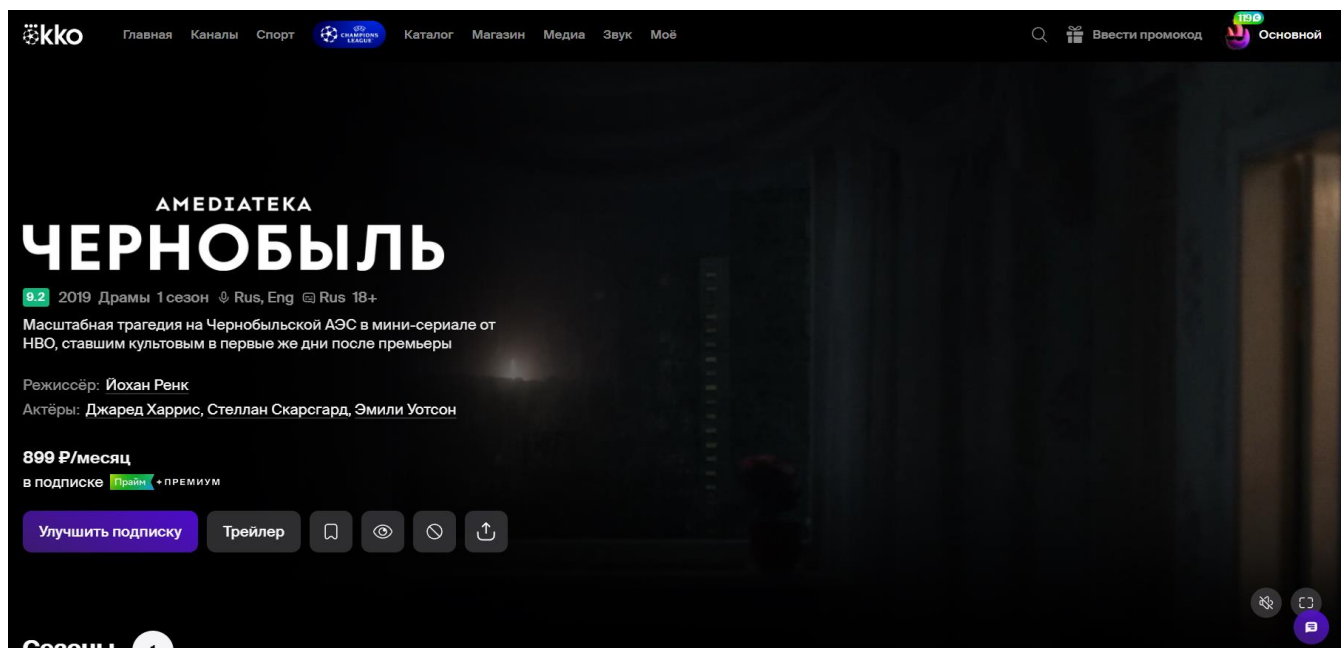


Рисунок 11 – Скриншот карточки платного фильма.

Модальное окно с предложением подписки – скриншот всплывающего окна после нажатия на «Смотреть». Результат: появляется окно, доступ к просмотру заблокирован.

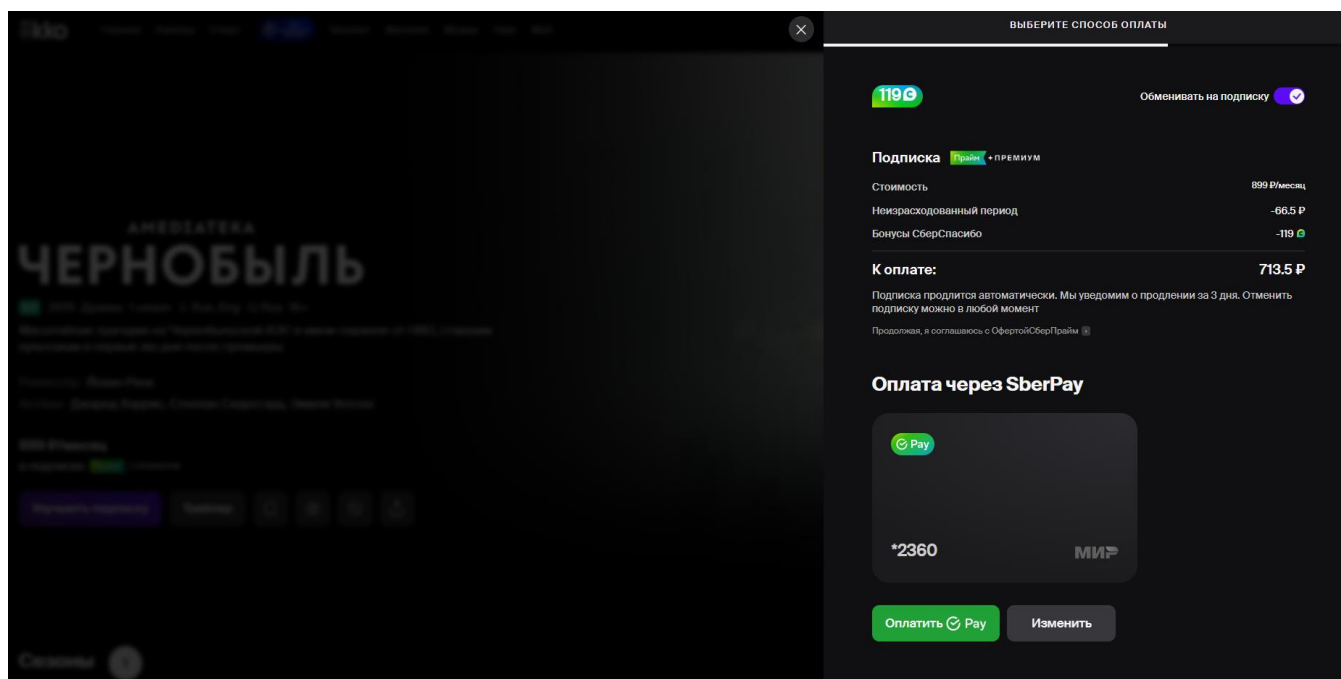


Рисунок 12 – Скриншот модального окна с предложением подписки

Закрытое модальное окно, возврат к карточке – скриншот исходной карточки.
Результат: просмотр не начат, система не позволяет продолжить без подписки.

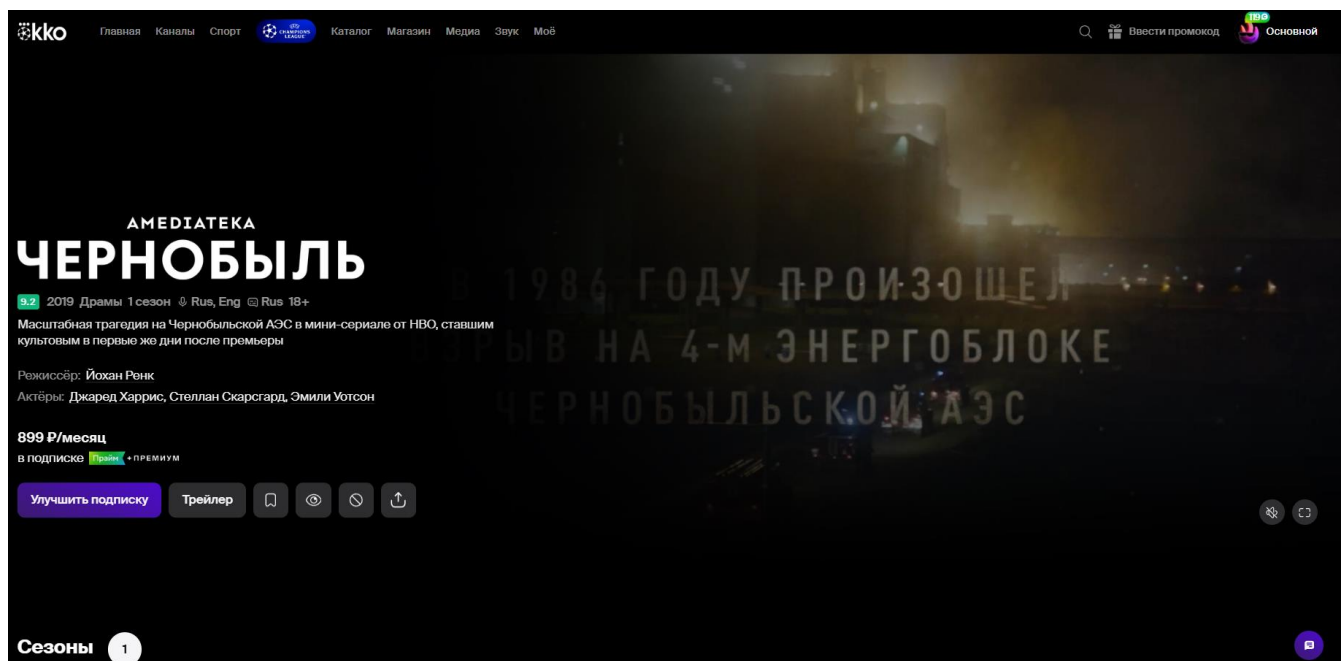


Рисунок 13 – Скриншот карточки после закрытия окна.

Running 'test3'

1. open on /serial/chernobyl OK
2. setWindowSize on 969x824 OK
3. click on css=.pLbFsg0W OK
4. mouseOver on css=.gmlAgFvx ._9vY2xAN OK
5. click on css=.gmlAgFvx ._9vY2xAN OK
6. runScript on window.scrollTo(0,0) OK
7. click on css=.css-tp7ox > span OK
8. mouseOver on css=.css-tp7ox > span OK
9. mouseOut on css=.css-tp7ox > span OK

'test3' completed successfully

Рисунок 14 – Выполнение первого сценария.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы были автоматизированы три тестовых сценария для сайта okko.tv с помощью инструмента Selenium IDE. Тесты успешно записаны, экспортированы в код на Python и выполнены. Позитивные тесты выполнены успешно, все действия воспроизведены корректно. Selenium IDE показал себя как удобный инструмент для быстрой записи тестов, особенно на этапе прототипирования. Экспорт в код позволяет интегрировать тесты в дальнейшую разработку